WEEK 2 · LESSON 10 OF 24 · HANDOUT

Grouping and Summarising Data

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=tRKeLrwfUgU — open the lesson online to watch inside the course.

🎓 **Go deeper:** freeCodeCamp: Data Analysis with Python (NumPy, Pandas, Matplotlib) — full course — https://www.youtube.com/watch?v=r-uOLxNrNk8

Your goal in this lesson: Answer business questions with `groupby`, aggregation functions and pivot tables, and combine tables with `merge`.

From rows to answers

200 rows of sales do not answer a manager's questions. Questions like *"Which district earns most?"* or *"How does MoMo use change by month?"* need **summaries**. Pandas does this with **split → apply → combine**:

1. **Split** the rows into groups (e.g. by district).
2. **Apply** a calculation to each group (e.g. sum of totals).
3. **Combine** the results into a new table.

Run the setup cell first:

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
n = 200
prices = {"Rice 5kg": 7500, "Cooking oil 1L": 3200, "Sugar 1kg": 1600,
          "Milk 1L": 800, "Airtime 1000": 1000}
products = rng.choice(list(prices), size=n, p=[0.2, 0.15, 0.2, 0.3, 0.15])
sales = pd.DataFrame({
    "date": pd.to_datetime("2026-01-01") + pd.to_timedelta(rng.integers(0, 181, n), unit="D"),
    "district": rng.choice(["Gasabo", "Kicukiro", "Nyarugenge"], size=n),
    "product": products,
    "quantity": rng.integers(1, 7, n),
    "payment": rng.choice(["MoMo", "Cash", "Card"], size=n, p=[0.55, 0.35, 0.10]),
})
sales["unit_price"] = sales["product"].map(prices)
sales["total"] = sales["quantity"] * sales["unit_price"]
sales = sales.sort_values("date").reset_index(drop=True)
```

groupby

```
by_district = sales.groupby("district")["total"].sum()
print(by_district)
print(by_district.sort_values(ascending=False))
```

Several measures at once with `agg`:

```
summary = sales.groupby("product").agg(
    sales_count=("total", "size"),
    units=("quantity", "sum"),
    income=("total", "sum"),
    average_sale=("total", "mean"),
).round(0)
print(summary.sort_values("income", ascending=False))
```

Common aggregation functions: `sum`, `mean`, `median`, `min`, `max`, `count`, `size`, `nunique`.

Grouping by more than one column

```
sales["month"] = sales["date"].dt.month
monthly = sales.groupby(["month", "payment"])["total"].sum().unstack()
print(monthly)
```

`unstack()` turns the second group level into columns — a quick way to make a readable table.

Pivot tables

If you know Excel pivot tables, Pandas has the same tool:

```
pivot = sales.pivot_table(index="district", columns="payment",
                          values="total", aggfunc="sum", margins=True)
print(pivot)
```

`margins=True` adds an **All** row and column with totals.

Shares and percentages

```
share = (sales.groupby("payment")["total"].sum() / sales["total"].sum() * 100).round(1)
print(share)
```

Combining tables with merge

Data often lives in separate tables. A **merge** joins them on a shared column:

```
managers = pd.DataFrame({
    "district": ["Gasabo", "Kicukiro", "Nyarugenge"],
    "manager": ["Aline", "Eric", "Grace"],
})
report = by_district.reset_index().merge(managers, on="district")
print(report)
```

This is the same idea as `VLOOKUP` in Excel, or a `JOIN` in SQL.

Asking good questions

A summary is only useful if it answers a real question. Before grouping, write the question in one sentence:

- *Which product brings the most income?* → group by product, sum total.
- *Do Card customers spend more per sale?* → group by payment, mean total.
- *Is business growing?* → group by month, sum total.

Key terms

- **groupby** — split rows into groups by a column.
- **Aggregation** — a calculation that turns many values into one (sum, mean…).
- **agg** — apply several named aggregations at once.
- **Pivot table** — a grid summary with groups in rows and columns.
- **merge** — join two tables on a shared column.

Remember this: Write the question first, then group, aggregate and combine. groupby, agg, pivot_table and merge turn raw rows into answers a manager can use.

Name: Date:

Activity

Task 1

Which month had the highest income? Which had the lowest?

Task 2

What is the average sale amount for each payment method? Do Card customers spend more?

Task 3

Make a pivot table of units sold: products in rows, districts in columns.

Task 4

Write three sentences a shop manager could act on, based on your results.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What are the three steps of groupby?
   A. Split, apply, combine
   B. Load, save, print
   C. Clean, train, test
   D. Sort, filter, plot
2. Which code gives total income per district?
   A. sales["district"].sum()
   B. sales.sort_values("district")
   C. sales.groupby("district")["total"].sum()
   D. sales.sum("district")
3. What does `agg(units=("quantity", "sum"))` create?
   A. An error
   B. A column named units with the sum of quantity for each group
   C. A new file
   D. A chart
4. What does `margins=True` add to a pivot table?
   A. Colours
   B. Missing values
   C. A title
   D. Total rows and columns (All)
5. What does `merge` do?
   A. Deletes duplicate columns
   B. Splits a table in two
   C. Joins two tables using a shared column
   D. Sorts a table
6. What should you do before writing a groupby?
   A. Draw a chart
   B. Write the business question in one sentence
   C. Nothing
   D. Delete half the data

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.